中南大学考试试卷答案（补考）

2008 -- 2009 学年 2 学期　　时间 110 分钟

算法分析与设计 课程 48 学时 3 学分 考试形式： 闭 卷

专业年级： 信安 0601-0602 总分 100 分，占总评成绩 70 %

注：此页不作答题纸，请将答案写在答题纸上

一、 基本概念题 （本大题 40 分）

1、 （6 分）

1 ）顺序结构将运算步骤的时间累计，简单运算只需要 1 个单位时间。（1 分）

2）选择结构：计算复杂的情况复杂度。（2 分）

3 ）循环结构：复杂度计量 =循环着次数 *循环体的时间（2 分）

4 ）函数调用：计算函数的执行时间（1 分）

2、设 T(n)=n，根据 T(n)= O(f （n）) 的定义，下列等式是否成立？ （4 分）

1） $T(n)= O(n^2)$ （√）

2） $O(n^2) = T(n)$ （×）

3） $T(n)= O(\log n)+ O(n)$ （√）

4） $T(n) = O(n) * O(\log n)$ （√）

3、与顺序查找算法相比，折半查找算法的时间复杂性有多大程度的降低？它是如何提高算法的效率的？ （6 分）

顺序查找的时间是 O(n) ，折半查找 O(log n) 降低了一个数量级 （2 分）

采用分治策略，每一次比较可以排除一半的数据。（4 分）

4、简述归并排序算法和快速排序算法的分治方法。（6 分）

1）归并排序的分治是将数组从中间分开，分别对前后来那个部分进行排序，将排序后的两个数组合并成整个数组的排序。 这样分治为递归过程，直到一个元素时返回。

2）快速排序的分治是选取分割元素， 以分割元素为界， 将数组分成两部分，一部分小于分割元素，一部分大于分割元素，分别对两部分排序。

5、一般背包问题的贪心算法可以获得最优解吗？物品的选择策略是什么？ （6 分）

按照 p[i]/w[i] ≥p[i+1]/w[i+1] 排序，选择当前利润 /重量比最大的物品，可以获得最优解，

6、Prim 算法和 Dijkstra 算法选择下一个节点的标准分别是什么？对于有负边的无向图， Prim 算法和 Dijkstra 算法还能保证获得最优解吗？ （6 分）

1）prim 算法的选择标准是选择当前与 T 连结边的代价最小的节点加入。

2）Dijkstra 算法的选择标准是在与 T 邻接的顶点 w 中，选择从 S 到 w 路径最短的顶点。

3）prim 算法用于有负边的图可以获得最优解， Dijkstra 算法不能获得最优解。

7、比较回溯法和分支限界法的搜索方式， 哪种方法更适合找最优解问题？ （6 分）

1）回溯法是在约束下带跳跃的深度优先搜索。

2）分枝限界是广度优先方式的按最小代价选择扩展节点，以上界函数对活节点进行限界的搜索。

3）分枝限界法更适合找最优解。

二、 分析算法的时间复杂性，需要写出分析过程 （本大题 20 分）

1、用分割元素 v 将有 n 个元素的数组分割成元素大于 v 和小于 v 的两部分，需要花多少时间（要讲出道理）。 （5 分）

至少需要对每个元素进行一次比较运算，运算时间是 O(n)。

2、如果修改归并排序算法，将数组分成 1/3 和 2/3 大小不等的两部分，分别排序后再归并，算法的最坏时间复杂度有什么变化？

设对 n 个元素排序的时间为 T(n)， 对两部分排序的时间分别为 T(n/3) 和，合并的时间为 n-1 ， 得到递归方程：

$T(n) = T(n/3)+ T(2n/3) + n-1 \quad n>3$ (2 分)

$O(1) \quad n \leq 3$

考虑 $n=3^k$

$T(n) = T(3^{k-1})+ T(2*3^{k-1}) +n-1$

$= T(3^{k-2})+2T(2*3^{k-2})+T(2^2*3^{k-2})+（n-1）+(n-2)$

$= T(3^{k-3})+3T(2^2*3^{k-3})+ 3T(2^2 3^{k-3})+T(2^3 3^{k-3})+（n-1）+(n-2) +(n-3)$

最后 $T(2^i 3^{k-i})=O(1)$ 时， $2^i 3^{k-i} \leq 3$

$T(n) \leq（n-1）+(n-2) +(n-3)+......+(n-(k-1))$

$=nk- （1+2+......+(k-1)）$

$\leq n\log_{3/2} n$ （3 分）

3、设函数 f1、f2 和 f3 的处理时间分别为 O(n) 、$O(n^2)$ 和 O(1)，分析下列流程的时间复杂性：

1) 基本结构

procedure A1(int n ， b) （4 分）

$T(n)=\max\{O(n),O(n^2)\}+n* O(1)$

$= O(n^2)$

2) 递归结构

设 A2 的时间为 T（n）

$T(n)= T(n-1)+O(1)\quad n>1$

$= O(1)\quad n \leq 3$ （3 分）

$T(n)=T(n-2)+2O(n)$

$=......$

$= T(1)+nO(n)$

$= O(n^2)$ （3 分）

三、 算法理解 （本大题 24 分）

1、在一个空间安排 n =5 个活动，开始时间和结束时间分别为。写出活动安排贪心算法的运行结果。

1）按照结束时间排序（ 3 分）

[8,10)1, [9, 11:30)3, [11:40,13)4 , [12,14)2, [13:30,15)5

2）可行解 1，4，5 （3 分）

2、写出 0/1 背包问题的动态规划方程，并简要说明。

f_i（X）$= \max\{f_{i-1}(X), \{f_{i-l}(X-w)+p_i$ 当 $X \geq wi\}$ （3 分）

f_i（X）是前 i 个物品，背包容积 X 子问题的最优值，

当第 i 个物品不选入， f_i（X）等于 $f_{i-1}(X)$ 前 i-1 个物品，背包容积 X 子问题的最优值，

当第 i 个物品不选入， 得利润 p_i，但前 i-1 个物品能使用背包为 X—w 。（3 分）

3、修改图的 m-着色的回溯算法，找到一个解，算法就结束。 （6 分）

```
Mcolor(n)
{k ← 1; x[k] ← 0;
 While k>0 do                          （2 分）
 { x[k] ← x[k]+1;
  while place(k)=false and x[k] ≤ m do
      x[k] ← x[k]+1
      if x[k] ≤ m then                 （2 分）
      if k=n then
       {print x
        Return                         （2 分）
       }
    else
        {k ← k+1
          x[k] ← 0}
  else k ← k-1
 }
```

4、用分支限界法解 0/1 背包问题，若物品 i 选入，则 x[i]=1，否则 x[i]=0。如何选用上下界函数？（6 分）

1）物品按照利润重量比排序，背包的剩余体积 cu，已得利润 s。（2 分）

2）下界估值函数：$-(s+\sum X[j]P[j])$，当 $\sum X[j]w[j]=cu$，$0\le X[j]\le 1, j=i,\cdots n$（2 分）

3）上界函数：$-(s+\sum X[j]P[j])$，当 $\sum X[j]w[j]\le cu, X[j]\in\{0,1\}$ $j=i,\cdots n$（2 分）

四、 算法设计 （本大题 16 分）

对于给定的无向图 G=(V,E)，分别设计具有下列功能的深度优先算法。

1) 判断图是否为连通图（8 分）

```
procedure DFS_Visit(G,u)
 {1  color[u]←Gray
   2  for each edge(u,v)  do
      2-1    if color[v]=White then
                   DFS_Visit(G,v)
   3  color[u]←Black;            （2 分）
}
procedure DFS(G)
 1 for each vertex u∈V  do
 2 color[u]←White
 3 f ←0
 3 for vertex each u∈V do
       if color[u]=White
          then { DFS_Visit(G,u)  （3 分）
                   f←f +1
                  }
 4 if  f=1 then print“Yes”    （3 分）
            else print“No ”
```

2) 判断图是否存在环（8 分）

```
procedure DFS_Visit(G,u)
 {1  color[u]←Gray
   2  for each edge(u,v)  do
      2-1 { if color[v]=White then
                   DFS_Visit(G,v)
      2-1  if  color[v]= Gray  then
             f← 1
             return
```

```
    color[u]←Black                    （2 分）
  }

procedure DFS(G)
  for each vertex u∈V   do
  color[u]←White
    f ←0
  for vertex each u∈V do
    {   if color[u]=White
            then { DFS_Visit(G, u)        （3 分）
                       if   f=1 then
                          { print “Yes”       （3 分）
                            return
                          }
                    }
    }
       print “No ”



                        }
  if   f=1 then print “Yes”       （3 分）
           else print “No ”
```